

Uniwersytet [Nazwa Uczelni]

Wydział [Nazwa Wydziału]

System do Rozliczania Wspólnych Wydatków

Dokumentacja projektu

Przedmiot: Aplikacje Internetowe
Autor: Krystian Oliwka
Kierunek / rok: Informatyka / 2025/2026
Prowadzący: dr hab. [Imię Nazwisko]

5 czerwca 2026

Spis treści

1	Opis projektu	1
1.1	Specyfikacja tematu	1
1.2	Technologie	1
2	Poziomy wymagań funkcjonalnych	2
2.1	Ocena 3.0 — podstawowy CRUD i administrator	2
2.2	Ocena 4.0 — rozszerzone uprawnienia	2
2.3	Ocena 5.0 — zaawansowana logika biznesowa	2
3	Aspekt projektowy bazy danych	4
3.1	Diagram ERD	4
3.2	Opis struktur tabel	4
3.3	Kod SQL tworzący bazę	5
4	Mechanizmy bazodanowe (triggery i funkcje)	7
4.1	Wymaganie techniczne	7
4.2	Triggery automatyzacji — <code>total_amount</code>	7
4.3	Trigger walidacyjny — członkostwo w grupie	8
4.4	Funkcja składowa <code>get_user_net_balance</code>	8
5	Założenia funkcjonalne i algorytmy	10
5.1	Mechanizmy CRUD w aplikacji	10
5.2	Algorytm 1: Równy podział rachunku	10
5.3	Algorytm 2: Bilans użytkownika w grupie	10
5.4	Przykład działania	11
6	Koncepcja dostępu zdalnego do bazy danych	12
6.1	Założenia	12
6.2	Konfiguracja połączenia	12
6.3	Zrzuty ekranu z aplikacji	12
7	Uruchomienie i testowanie	15
7.1	Wymagania środowiskowe	15
7.2	Kroki uruchomienia	15
7.3	Konta testowe	15
7.4	Scenariusz testowy	15
8	Struktura projektu	17

A Spis plików migracji

18

Rozdział 1

Opis projektu

1.1 Specyfikacja tematu

Zaprojektowano i zaimplementowano system webowy do sprawiedliwego rozliczania wspólnych wydatków w grupach (np. wycieczki, wspólne mieszkanie, imprezy). System rozwiązuje problem nieczytelności arkuszy kalkulacyjnych przy dużej liczbie uczestników i transakcji.

Główne funkcjonalności

- tworzenie grup rozliczeniowych i zapraszanie członków,
- dodawanie rachunków z wyborem płatnika spośród uczestników,
- automatyczny podział kosztów na uczestników (`bill_splits`),
- rozbijanie rachunku na pozycje z paragonu (`bill_items`) z przypisaniem do osób,
- automatyczne bilansowanie należności (kto komu ile jest winien),
- system ról użytkowników (`user` / `admin`),
- triggerzy bazodanowe wspierające logikę systemu.

1.2 Technologie

Tabela 1.1: Stack technologiczny

Warstwa	Technologia
Język / framework	PHP 8.2, Laravel 12
Baza danych	MySQL / MariaDB (produkcja), SQLite (dev)
Frontend	Blade, Tailwind CSS, Vite
Autentykacja	Laravel Breeze

Rozdział 2

Poziomy wymagań funkcjonalnych

2.1 Ocena 3.0 — podstawowy CRUD i administrator

System realizuje operacje CRUD na zasobach aplikacji. Administrator zarządza zasobami systemowymi.

Tabela 2.1: Mapa operacji CRUD — ocena 3.0

Zasób	C	R	U	D
Grupy rozliczeniowe	✓	✓	✓	✓
Rachunki / wydatki	✓	✓	—	✓
Użytkownicy (panel admina)	—	✓	✓	✓

Administrator (`role = admin`) zarządza użytkownikami w panelu `/admin/users`.

2.2 Ocena 4.0 — rozszerzone uprawnienia

- kolumna `users.role` przyjmuje wartości `user` lub `admin`,
- zwykły użytkownik zarządza własnymi grupami (tworzenie, edycja, usuwanie, dodawanie członków),
- administrator widzi wszystkie grupy, może je edytować i usuwać,
- administrator zarządza profilami i rolami innych użytkowników,
- kontrola dostępu realizowana przez `AdminMiddleware`.

2.3 Ocena 5.0 — zaawansowana logika biznesowa

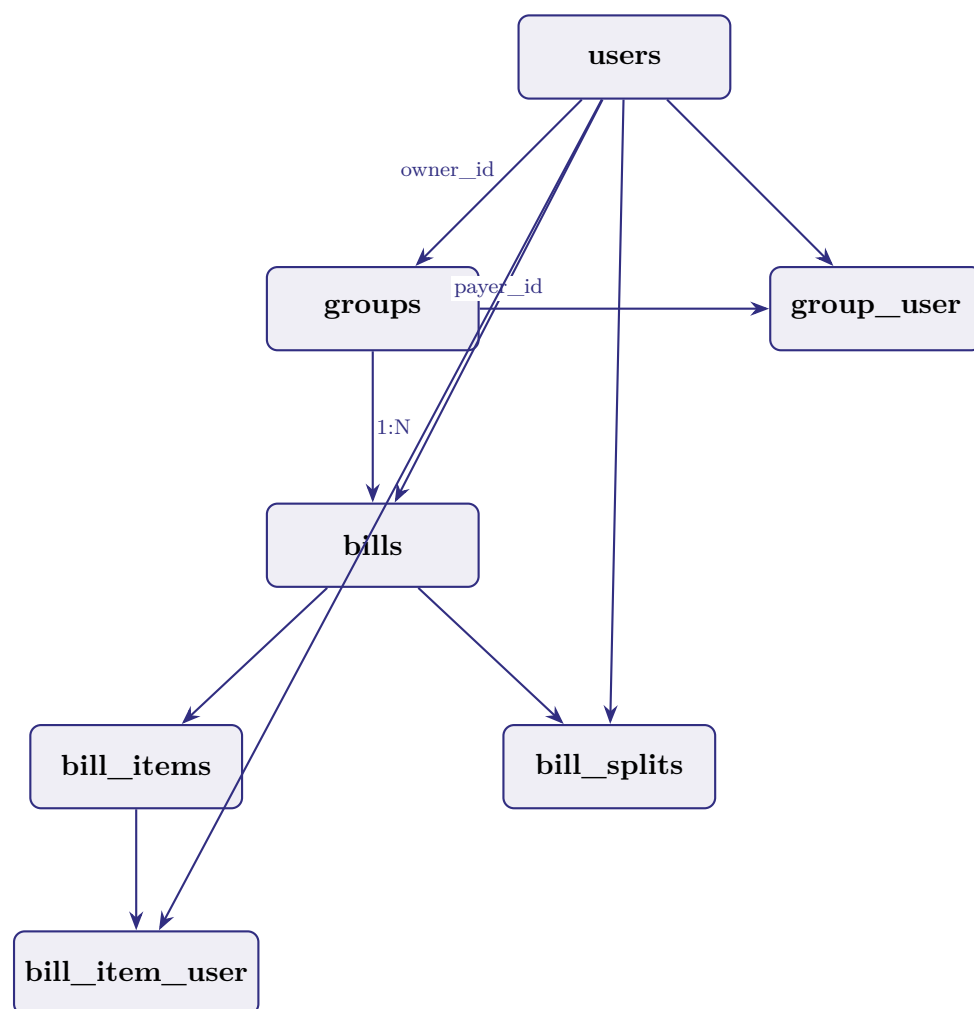
- **Wielu płatników** — przy dodawaniu rachunku wybiera się płatnika z listy członków grupy.
- **Podział kosztów** — tabela `bill_splits`; przy dodaniu rachunku tworzone są rekordy z równym udziałem.
- **Pozycje z paragonu** — tabele `bill_items` i `bill_item_user`.

- **Bilansowanie** — algorytm: $\text{saldo} = \text{zapłacone} - \text{należności}$; na MySQL wspierane funkcją składową.
- **Triggery DB** — automatyczna aktualizacja `groups.total_amount` oraz walidacja członkostwa w grupie.

Rozdział 3

Aspekt projektowy bazy danych

3.1 Diagram ERD



Rysunek 3.1: Diagram encji i powiązań (ERD)

3.2 Opis struktur tabel

Tabela	Opis i kluczowe kolumny
users	Konta użytkowników. Kolumny: id, name, email, password, role (user/admin).
groups	Grupy rozliczeniowe. owner_id → users.id. total_amount utrzymywane przez trigger.
group_user	Tabela łącznikowa M:N — członkostwo w grupie.
bills	Rachunki grupowe. payer_id wskazuje płatnika. amount, description, date.
bill_splits	Udział każdego użytkownika w danym rachunku. is_paid — czy spłacone.
bill_items	Pozycje z paragonu (np. Pizza, Napój). price, quantity.
bill_item_user	Przypisanie pozycji paragonu do użytkowników (M:N).

Tabela 3.1: Opis tabel bazy danych

3.3 Kod SQL tworzący bazę

```

1 CREATE TABLE users (
2     id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
3     name VARCHAR(255) NOT NULL,
4     email VARCHAR(255) NOT NULL UNIQUE,
5     role VARCHAR(255) NOT NULL DEFAULT 'user',
6     password VARCHAR(255) NOT NULL,
7     email_verified_at TIMESTAMP NULL,
8     remember_token VARCHAR(100) NULL,
9     created_at TIMESTAMP NULL,
10    updated_at TIMESTAMP NULL
11 );

```

Listing 3.1: Tabela użytkowników

```

1 CREATE TABLE groups (
2     id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
3     name VARCHAR(255) NOT NULL,
4     owner_id BIGINT UNSIGNED NOT NULL,
5     total_amount DECIMAL(12,2) DEFAULT 0.00,
6     created_at TIMESTAMP NULL,
7     updated_at TIMESTAMP NULL,
8     FOREIGN KEY (owner_id) REFERENCES users(id) ON DELETE CASCADE
9 );

```

Listing 3.2: Grupy rozliczeniowe

```

1 CREATE TABLE bills (
2     id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
3     group_id BIGINT UNSIGNED NOT NULL,
4     payer_id BIGINT UNSIGNED NOT NULL,
5     description VARCHAR(255) NOT NULL,
6     amount DECIMAL(10,2) NOT NULL,

```



```
7      date DATE NOT NULL,  
8      created_at TIMESTAMP NULL,  
9      updated_at TIMESTAMP NULL,  
10     FOREIGN KEY (group_id) REFERENCES groups(id) ON DELETE  
      CASCADE,  
11     FOREIGN KEY (payer_id) REFERENCES users(id) ON DELETE CASCADE  
12 );
```

Listing 3.3: Rachunki

```
1 CREATE TABLE bill_splits (  
2     id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
3     bill_id BIGINT UNSIGNED NOT NULL,  
4     user_id BIGINT UNSIGNED NOT NULL,  
5     amount DECIMAL(10,2) NOT NULL,  
6     is_paid BOOLEAN DEFAULT FALSE,  
7     created_at TIMESTAMP NULL,  
8     updated_at TIMESTAMP NULL,  
9     FOREIGN KEY (bill_id) REFERENCES bills(id) ON DELETE CASCADE,  
10    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE  
11 );  
12  
13 CREATE TABLE bill_items (  
14     id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
15     bill_id BIGINT UNSIGNED NOT NULL,  
16     name VARCHAR(255) NOT NULL,  
17     price DECIMAL(10,2) NOT NULL,  
18     quantity INT DEFAULT 1,  
19     created_at TIMESTAMP NULL,  
20     updated_at TIMESTAMP NULL,  
21     FOREIGN KEY (bill_id) REFERENCES bills(id) ON DELETE CASCADE  
22 );  
23  
24 CREATE TABLE bill_item_user (  
25     id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
26     bill_item_id BIGINT UNSIGNED NOT NULL,  
27     user_id BIGINT UNSIGNED NOT NULL,  
28     FOREIGN KEY (bill_item_id) REFERENCES bill_items(id) ON  
      DELETE CASCADE,  
29     FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE  
30 );
```

Listing 3.4: Podział kosztów i pozycje paragonu

Rozdział 4

Mechanizmy bazodanowe (triggery i funkcje)

4.1 Wymaganie techniczne

Projekt implementuje **obowiązkowe triggery bazodanowe** w dialekcie MySQL. Kod znajduje się w migracji `2026_05_14_163307_add_trigger_and_total_to_groups.php`.

4.2 Triggery automatyzacji — `total_amount`

Trzy triggery na tabeli `bills` automatycznie aktualizują pole `groups.total_amount`:

```
1 CREATE TRIGGER update_group_total_after_bill_insert
2 AFTER INSERT ON bills FOR EACH ROW
3 BEGIN
4     UPDATE groups
5     SET total_amount = total_amount + NEW.amount
6     WHERE id = NEW.group_id;
7 END;
```

Listing 4.1: Trigger po INSERT na bills

```
1 CREATE TRIGGER update_group_total_after_bill_update
2 AFTER UPDATE ON bills FOR EACH ROW
3 BEGIN
4     UPDATE groups
5     SET total_amount = total_amount - OLD.amount + NEW.amount
6     WHERE id = NEW.group_id;
7 END;
```

Listing 4.2: Trigger po UPDATE na bills

```
1 CREATE TRIGGER update_group_total_after_bill_delete
2 AFTER DELETE ON bills FOR EACH ROW
3 BEGIN
4     UPDATE groups
5     SET total_amount = total_amount - OLD.amount
6     WHERE id = OLD.group_id;
7 END;
```

Listing 4.3: Trigger po DELETE na bills

4.3 Trigger walidacyjny — członkostwo w grupie

Przed przypisaniem pozycji paragonu do użytkownika trigger sprawdza, czy użytkownik należy do grupy:

```

1 CREATE TRIGGER validate_user_in_group_before_item_assign
2 BEFORE INSERT ON bill_item_user FOR EACH ROW
3 BEGIN
4     DECLARE v_group_id INT;
5     DECLARE v_is_member INT DEFAULT 0;
6
7     SELECT b.group_id INTO v_group_id
8     FROM bill_items bi
9     JOIN bills b ON bi.bill_id = b.id
10    WHERE bi.id = NEW.bill_item_id;
11
12    SELECT COUNT(*) INTO v_is_member
13    FROM group_user
14    WHERE group_id = v_group_id AND user_id = NEW.user_id;
15
16    IF v_is_member = 0 THEN
17        SIGNAL SQLSTATE '45000'
18        SET MESSAGE_TEXT = 'Błąd Walidacji DB: Użytkownik nie
19                             należy do tej grupy!';
20    END IF;
21 END;
```

Listing 4.4: Walidacja przy INSERT na bill_item_user

4.4 Funkcja składowa get_user_net_balance

```

1 CREATE FUNCTION get_user_net_balance(p_user_id INT, p_group_id
2   INT)
3 RETURNS DECIMAL(10,2) DETERMINISTIC
4 BEGIN
5     DECLARE v_total_paid DECIMAL(10,2) DEFAULT 0.00;
6     DECLARE v_total_owed DECIMAL(10,2) DEFAULT 0.00;
7
8     SELECT IFNULL(SUM(amount), 0.00) INTO v_total_paid
9     FROM bills
10    WHERE payer_id = p_user_id AND group_id = p_group_id;
11
12    SELECT IFNULL(SUM(bs.amount), 0.00) INTO v_total_owed
13    FROM bill_splits bs
14    JOIN bills b ON bs.bill_id = b.id
15    WHERE bs.user_id = p_user_id AND b.group_id = p_group_id;
```

```
15  
16     RETURN v_total_paid - v_total_owed;  
17 END;
```

Listing 4.5: Obliczanie salda netto użytkownika w grupie

Funkcja jest wywoływana w modelu `Group::getBalances()` przy połączeniu z MySQL.

Rozdział 5

Założenia funkcjonalne i algorytmy

5.1 Mechanizmy CRUD w aplikacji

Tabela 5.1: Mapa kontrolerów i tras

Zasób	Kontroler	Trasy
Grupy	GroupController	groups.*
Rachunki	BillController	bills.store, bills.destroy
Pozycje paragonu	BillItemController	bill-items.store
Użytkownicy (admin)	Admin\UserController	admin.users.*
Profil	ProfileController	profile.*

5.2 Algorytm 1: Równy podział rachunku

```
1 WEJSCIE: rachunek R o kwocie K, grupa G z N członkami
2 WYJSCIE: rekordy w tabeli bill_splits
3
4 udział = K / N
5 DLA KAZDEGO członka C z grupy G:
6     UTWÓRZ bill_split(bill_id=R, user_id=C, amount=udział)
7     JEŚLI C jest płatnikiem:
8         ustaw is_paid = TRUE
```

Listing 5.1: Pseudokod — tworzenie bill_splits

Implementacja: `BillController::createEqualSplits()`.

5.3 Algorytm 2: Bilans użytkownika w grupie

```
1 WEJSCIE: user_id U, group_id G
2 WYJSCIE: saldo S
3
4 zaplacone = SUMA(bills.amount WHERE payer_id=U AND group_id=G)
5 naleznosci = SUMA(bill_splits.amount WHERE user_id=U AND bill IN
  G)
```

```
6 S = zaplacone - naleznosci
7
8 JESLI S > 0:   inni winni U kwote S
9 JESLI S < 0:   U winien innym kwote |S|
10 JESLI S = 0:   rozliczony
```

Listing 5.2: Pseudokod — obliczanie salda

Implementacja: `Group::getBalances()` oraz funkcja SQL `get_user_net_balance()`.

5.4 Przykład działania

Grupa 3-osobowa. Krystian płaci 600 zł za obiad. Każdy dostaje udział 200 zł:

Osoba	Zapłaciła	Saldo
Krystian	600 zł	+400 zł
Adam	0 zł	−200 zł
Ewa	0 zł	−200 zł

Tabela 5.2: Przykładowe salda po rachunku 600 zł

Rozdział 6

Koncepcja dostępu zdalnego do bazy danych

6.1 Założenia

- Baza MySQL/MariaDB hostowana lokalnie (XAMPP, Laragon) lub na serwerze zdalnym.
- Dostęp administracyjny przez phpMyAdmin, HeidiSQL lub MySQL Workbench.
- Aplikacja łączy się z bazą przez PDO — konfiguracja w pliku `.env`.

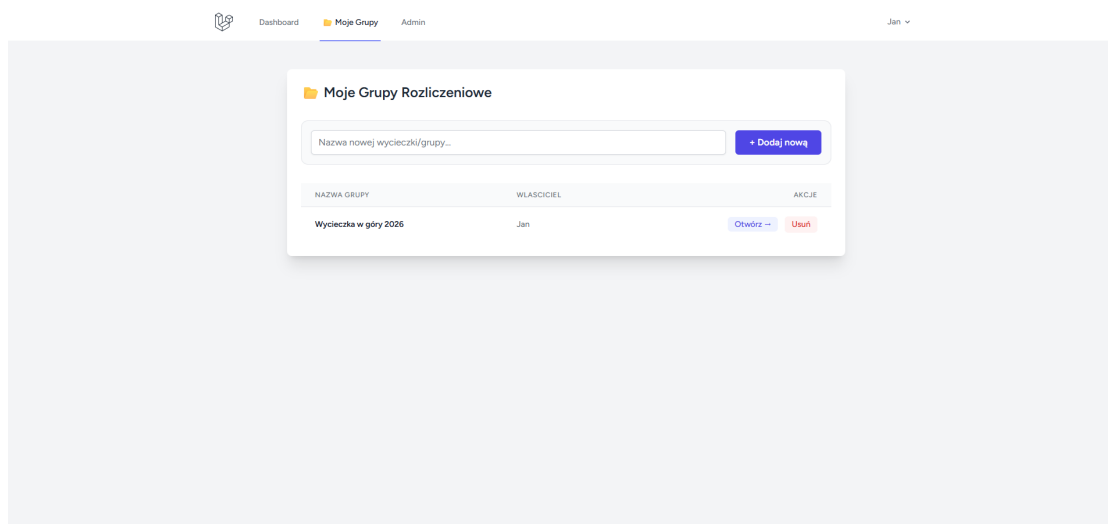
6.2 Konfiguracja połączenia

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=rozliczenia
DB_USERNAME=root
DB_PASSWORD=
```

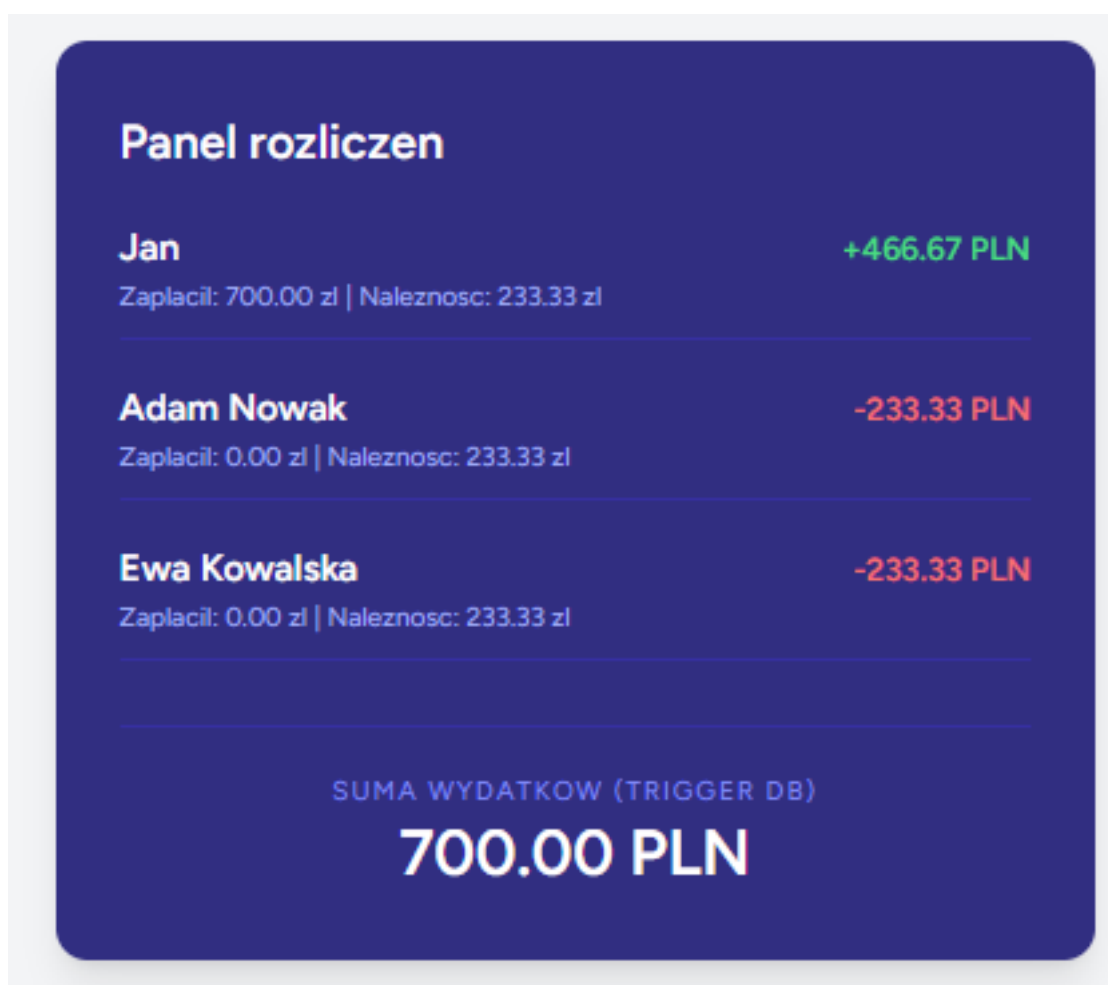
Listing 6.1: Fragment pliku `.env`

6.3 Zrzuty ekranu z aplikacji

Poniżej przedstawiono zrzuty ekranu z działającej aplikacji webowej. Triggery i funkcja składowa są zaimplementowane w kodzie SQL (rozdział 4) — w trybie deweloperskim projekt korzysta z SQLite (`database/database.sqlite`), na produkcji z MySQL.



Rysunek 6.1: Lista grup rozliczeniowych użytkownika



Rysunek 6.2: Panel rozliczeń — salda członków grupy

Członkowie grupy

Jan LIDER

Adam Nowak

Ewa Kowalska

E-mail znajomego...

Dodaj do grupy

Panel rozliczeń

Jan +466.67 PLN

Zapłacił: 700.00 zł | Należność: 233.33 zł

Adam Nowak -233.33 PLN

Zapłacił: 0.00 zł | Należność: 233.33 zł

Ewa Kowalska -233.33 PLN

Zapłacił: 0.00 zł | Należność: 233.33 zł

SUMA WYDATKÓW (TRIGGER DB)

700.00 PLN

Dodaj wydatek

Za co?

Kwota

Jan

Dodaj

HISTORIA ROZLICZEN

Obiad i napoje Usun

Płatnik: Jan

600.00 PLN

Podział (bill_splits): Jan: 200.00 zł, Adam Nowak: 200.00 zł, Ewa Kowalska: 200.00 zł

Pozycje z paragonu:

Duża Pizza (80.00 zł x1) - przypisane: brak

[Dodaj pozycje z paragonu](#)

Paliwo Usun

Płatnik: Jan

100.00 PLN

Podział (bill_splits): Jan: 33.33 zł, Adam Nowak: 33.33 zł, Ewa Kowalska: 33.33 zł

[Dodaj pozycje z paragonu](#)

Rysunek 6.3: Historia rachunków z podziałem kosztów (bill_splits) i pozycjami paragonu

Dashboard Moje Grupy **Admin** Jan

Panel administratora - uzytkownicy

ID/E	EMAIL	ROLA	AKCJE
Adam Nowak	adam@example.com	user	Usun
Ewa Kowalska	ewa@example.com	user	Usun
Jan	jan@example.com	admin	
Jan Hacker	hacker@example.com	user	Usun
John	doe@gmail.com	user	Usun

Rysunek 6.4: Panel administratora — zarządzanie użytkownikami i rolami

Rozdział 7

Uruchomienie i testowanie

7.1 Wymagania środowiskowe

PHP 8.2+, Composer, Node.js. Instrukcja szczegółowa w pliku JAK_URUCHOMIC.md.

7.2 Kroki uruchomienia

1. `composer install` i `npm install`
2. Skopiuj `.env.example` → `.env`, wygeneruj klucz: `php artisan key:generate`
3. Uruchom migracje: `php artisan migrate:fresh --seed`
4. Zbuduj frontend: `npm run build`
5. Uruchom serwer: `php artisan serve` → <http://127.0.0.1:8000>

7.3 Konta testowe

Rola	E-mail	Hasło
admin	krystian@example.com	password
user	adam@example.com	password
user	ewa@example.com	password

Tabela 7.1: Konta demonstracyjne po seedzie

7.4 Scenariusz testowy

1. Zaloguj się jako administrator (`krystian@example.com`).
2. Przejdź do *Moje Grupy* → otwórz *Wycieczka w góry 2026*.
3. Sprawdź panel rozliczeń — salda członków.
4. Dodaj nowy wydatek z wybranym płatnikiem.

5. Dodaj pozycję z paragonu i przypisz ją do osób.
6. Wejdź w panel *Admin* — zmień rolę użytkownika.
7. Wyloguj się, zaloguj jako `adam@example.com` — sprawdź widoczność grupy.

Rozdział 8

Struktura projektu

```
1 AplikacjeInternetoweProjekt/  
2   Dokumentacja/  
3     dokumentacja.pdf          -- dokumentacja do oddania  
4     dokumentacja.tex          -- zrodlo LaTeX  
5     screenshots/              -- zrzuty ekranu  
6   ProjektAplikacje/           -- aplikacja Laravel  
7     app/Http/Controllers/  
8     app/Http/Middleware/  
9     app/Models/  
10    database/migrations/  
11    database/seeder/  
12    resources/views/  
13    routes/web.php
```

Listing 8.1: Struktura repozytorium

Dodatek A

Spis plików migracji

Plik migracji	Zawartość
create_users_table	Tabela użytkowników
add_role_to_users_table	Kolumna role
create_groups_table	Grupy + total_amount
create_group_user_table	Członkostwo M:N
create_bills_table	Rachunki
create_bill_items_table	Pozycje + bill_item_user
create_bill_splits_table	Podział kosztów
add_trigger_and_total_to_groups	Triggery + funkcja SQL